

Introduction to COBOL

COBOL

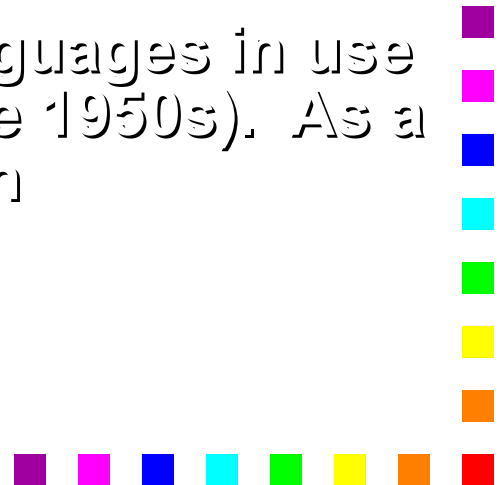
COBOL is an acronym which stands for
Common **B**usiness **O**riented **L**anguage.

The name indicates the target area of COBOL applications.

COBOL is used for developing business, typically file-oriented, applications.

It is not designed for writing systems programs. You would not develop an operating system or a compiler using COBOL.

COBOL is one of the oldest computer languages in use (it was developed around the end of the 1950s). As a result it has some idiosyncracies which programmers may find irritating.



COBOL idiosyncracies

One of the design goals was to make the language as English-like as possible. As a consequence

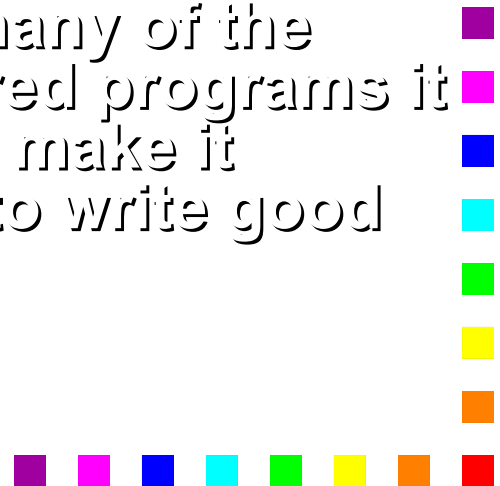
the COBOL reserved word list has grown to hundreds of entries.

COBOL uses structural constructs that resemble English prose such as sections and paragraphs.

As a result COBOL programs

Some implementations require the program text to adhere to certain, archaic, formatting restrictions.

Although modern COBOL has introduced many of the constructs required to write well structured programs it also still retains elements which, if used, make it difficult, and in some cases impossible, to write good programs.



The COBOL Metalanguage - Syntax Notation

Words in uppercase are reserved words.

When underlined they must be used
which they are a part is underlined.

When they are not underlined they are optional
and are optional. If used they must be used.

Words in mixed case represent names which will be devised by the programmer.

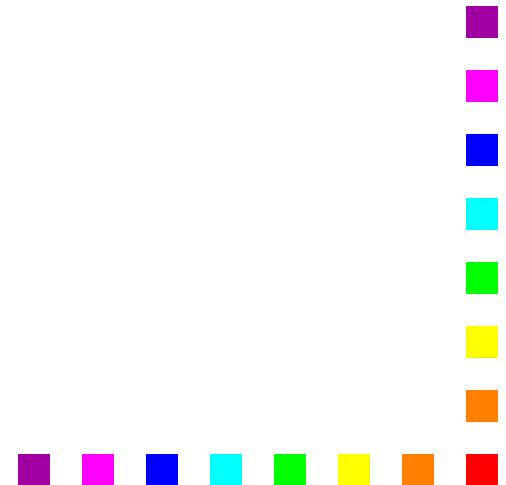
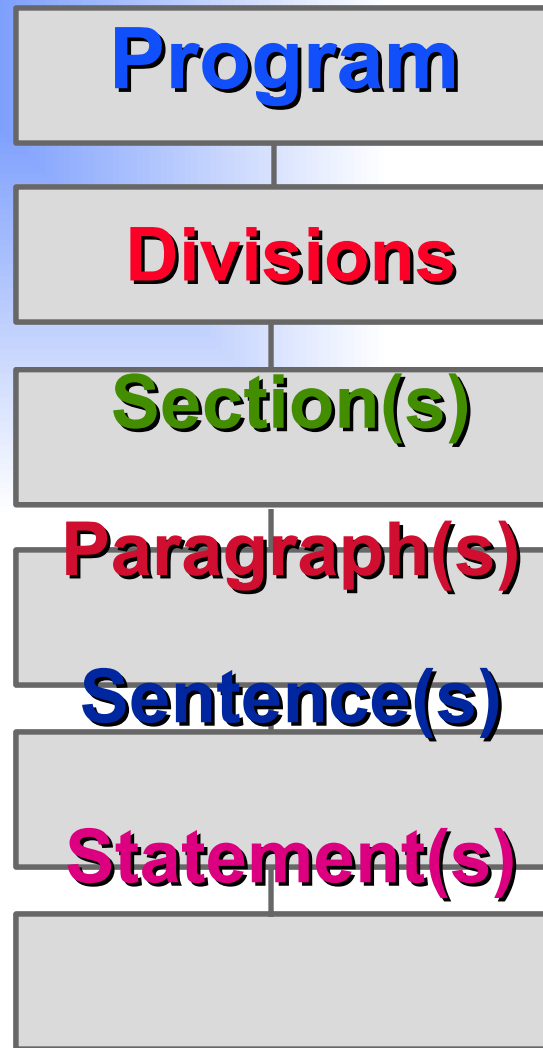
When material is enclosed in braces { } a choice must be made from the options within the braces.

Material is enclosed in square brackets [] indicates that the material is an option which may be included or omitted as required.

The ellipsis symbol '...' indicates that the preceding syntax element may be repeated at the programmer's discretion.



Structure of COBOL programs.



The Four Divisions.

DIVISIONS are used to identify the principal components of the program text. There are four **DIVISIONS** in all.

- ◆ **IDENTIFICATION DIVISION.**
- ◆ **ENVIRONMENT DIVISION.**
- ◆ **DATA DIVISION.**
- ◆ **PROCEDURE DIVISION.**

Although some of the divisions may be omitted the sequence in which the **DIVISIONS** are specified is fixed and **must** follow the pattern shown above.



Functions of the four divisions.

The **IDENTIFICATION DIVISION** is used to supply information about the program to the programmer and to the compiler.

The **ENVIRONMENT DIVISION** describes to the compiler the environment in which the program will run.

As the name suggests, the **DATA DIVISION** is used to provide the descriptions of most of the data to be processed by the program.

The **PROCEDURE DIVISION** contains the description of the algorithm which will manipulate the data previously described. Like other languages COBOL provides a means for specifying sequence, selection and iteration constructs.



COBOL Program Text Structure

IDENTIFICATION DIVISION.

Program Details

DATA DIVISION.

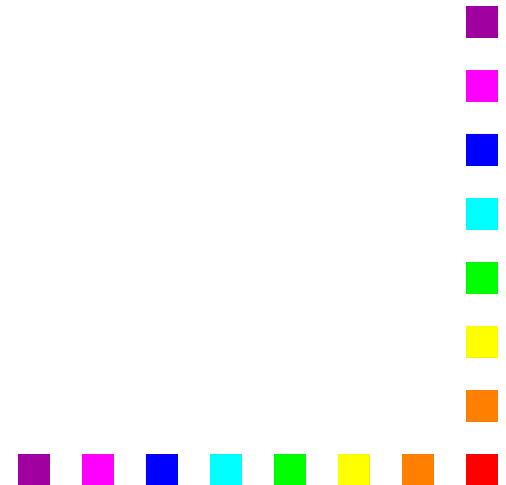
Data Descriptions

PROCEDURE DIVISION.

Algorithm Description

NOTE

The keyword DIVISION and a 'full-stop' is used in every case.



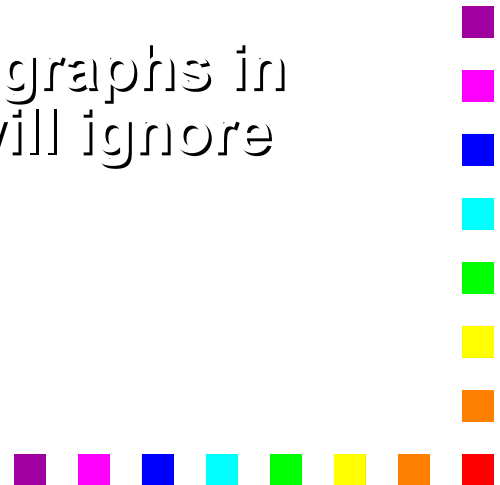
IDENTIFICATION DIVISION.

The purpose of the IDENTIFICATION DIVISION is to provide information about the program to the **programmer** and to the **compiler**.

Most of the entries in the IDENTIFICATION DIVISION are directed at the **programmer** and are treated by the compiler as **comments**.

An exception to this is the **PROGRAM-ID** clause. Every COBOL program must have a PROGRAM-ID. It is used to enable the compiler to identify the program.

There are several other informational paragraphs in the IDENTIFICATION DIVISION but we will ignore them for the moment.



The IDENTIFICATION DIVISION Syntax.

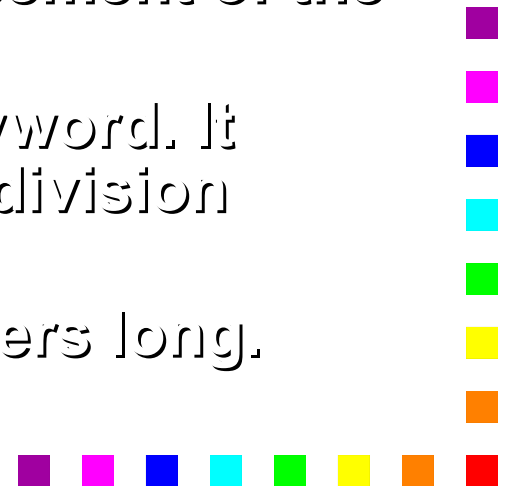
The IDENTIFICATION DIVISION has the following structure

```
IDENTIFICATION DIVISION.  
PROGRAM-ID. NameOfProgram.  
[AUTHOR. YourName. ]
```

The keywords **IDENTIFICATION DIVISION** represent the division header and signal the commencement of the program text.

The paragraph name **PROGRAM-ID** is a keyword. It must be specified immediately after the division header.

The program name can be up to 30 characters long.



The IDENTIFICATION DIVISION Syntax.

The IDENTIFICATION DIVISION has the following structure

```
IDENTIFICATION DIVISION.  
PROGRAM-ID. NameOfProgram.  
[AUTHOR. YourName. ]
```

```
$ SET SOURCEFORMAT"FREE"  
IDENTIFICATION DIVISION.  
PROGRAM-ID. SequenceProgram.  
AUTHOR. Michael Coughlan.
```

The keywords IDENTIFICATION DIVISION represent the division header and signal the commencement of the program text.

The paragraph name PROGRAM-ID is a keyword. It must be specified immediately after the division header.

The program name can be up to 30 characters long.



The DATA DIVISION.

The **DATA DIVISION** is used to describe most of the data that a program processes.

The **DATA DIVISION** is divided into two main sections;

FILE SECTION.

WORKING-STORAGE SECTION.

The **FILE SECTION** is used to describe most of the data that is sent to, or comes from, the computer's peripherals.

The **WORKING-STORAGE SECTION** is used to describe the general variables used in the program.



DATA DIVISION Syntax

The DATA DIVISION has the following structure

DATA DIVISION.

[FILE SECTION.

File Section entries.]

[WORKING-STORAGE SECTION.

WS entries.]

```
      $ SET SOURCEFORMAT"FREE"
IDENTIFICATION DIVISION.
PROGRAM-ID.    Sequence-Program.
AUTHOR.       Michael Coughlan.

DATA DIVISION.
WORKING-STORAGE SECTION.
01  Num1                PIC 9  VALUE ZEROS.
01  Num2                PIC 9  VALUE ZEROS.
01  Result              PIC 99 VALUE ZEROS.
```



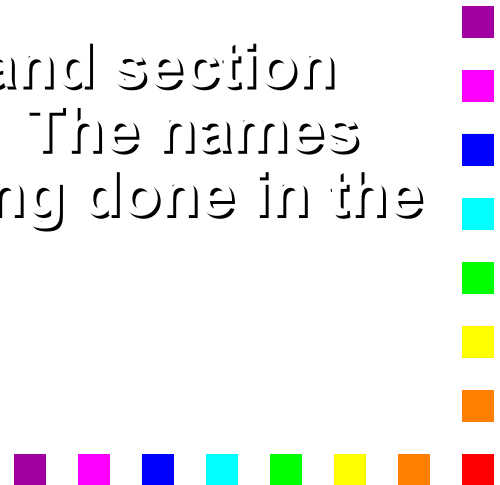
The PROCEDURE DIVISION.

The PROCEDURE DIVISION is where all the **data** described in the DATA DIVISION is processed and produced. It is here that the programmer describes his algorithm.

The PROCEDURE DIVISION is hierarchical in structure and consists of Sections, Paragraphs, Sentences and Statements.

Only the Section is optional. There must be **at least one** paragraph, sentence and statement in the PROCEDURE DIVISION.

In the PROCEDURE DIVISION paragraph and section names are chosen by the programmer. The names used should reflect the processing being done in the paragraph or section.

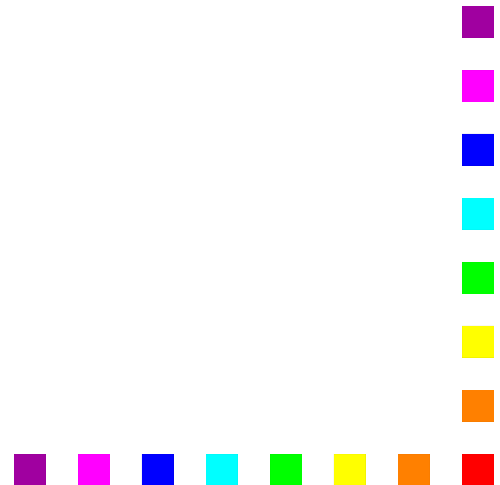


Sections

A **section** is a block of code made up of one or more **paragraphs**.

A section begins with the section name and ends where the next section name is encountered or where the program text ends.

A section name consists of a name devised by the programmer or defined by the language followed by the word `SECTION` followed by a full stop.



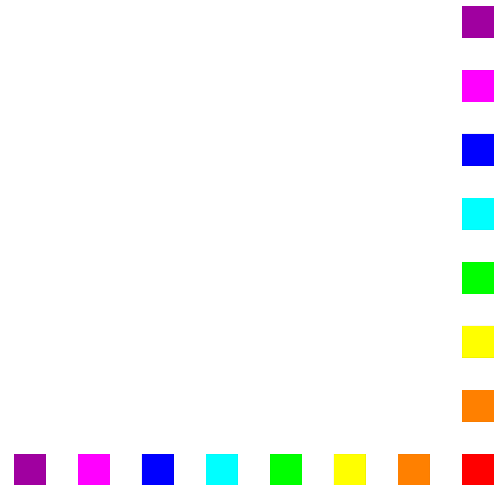
Paragraphs

Each section consists of one or more paragraphs.

A **paragraph** is a block of code made up of one or more **sentences**.

A paragraph begins with the paragraph name and ends with the next paragraph or section name or the end of the program text.

The paragraph name consists of a name devised by the programmer or defined by the language followed by a full stop.



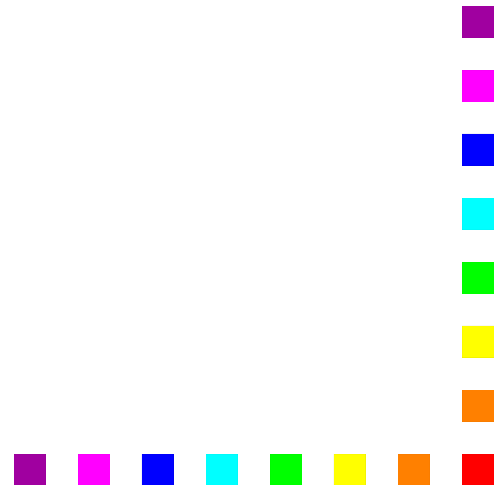
Sentences and Statements.

A paragraph consists of one or more sentences.

A **sentence** consists of one or more **statements** and is terminated by a full stop.

```
MOVE .21 TO VatRate  
  COMPUTE VatAmount =  
    DISPLAY "Enter name " W  
      ACCEPT StudentName
```

A **statement** consists of a COBOL **verb** and an **operand** or operands.



A Full COBOL program.

```
      $ SET SOURCEFORMAT"FREE"  
IDENTIFICATION DIVISION.  
PROGRAM-ID.  SequenceProgram.  
AUTHOR.  Michael Coughlan.  
  
DATA DIVISION.  
WORKING-STORAGE SECTION.  
01  Num1          PIC 9  VALUE ZEROS.  
01  Num2          PIC 9  VALUE ZEROS.  
01  Result        PIC 99 VALUE ZEROS.  
  
PROCEDURE DIVISION.  
CalculateResult.  
    ACCEPT Num1.  
    ACCEPT Num2.  
    MULTIPLY Num1 BY Num2 GIVING Result.  
    DISPLAY "Result is = ", Result.  
    STOP RUN.
```



The minimum COBOL program.

```
      $ SET SOURCEFORMAT"FREE"  
IDENTIFICATION DIVISION.  
PROGRAM-ID.    SmallestProgram.  
  
PROCEDURE DIVISION.  
DisplayPrompt.  
    DISPLAY "I did it".  
    STOP RUN.
```

